

WEMOVE 项目沟通记录

徐熠潘 ↔ GPT

从网站目标、架构边界到需求与可维护性

整理时间：2026 年 9 月 8 日 | 对话来源：抽象问题讨论、细节问题讨论

本记录根据《徐熠潘讨论记录汇总》整理。对话保留原始讨论中的事实、技术细节和结论；相近问题合并为主题，成员与 AI 的称谓统一为“徐熠潘”和“GPT”。

目录

1	对话记录	2
1.1	从“好网站”到本地演示的最小架构	2
1.2	网站设计流程、信息架构与绘图	3
1.3	接口契约与组件协作	4
1.4	WEMOVE 面向谁、完成什么、项目目标是什么	5
1.5	需求文档与 SRS 模板	6
1.6	可维护性、模块边界与技术栈	7

1 对话记录

1.1 从“好网站”到本地演示的最小架构

 徐熠潘

一个完整且优秀的网站应该具备哪些特质？除了页面和视觉，还需要哪些组成部分？

 GPT

完整不等于页面多，优秀也不等于视觉精致。核心是让目标用户稳定完成目标，并且系统可维护、可演进。质量可以从有效性、易用性、可靠性与性能、安全与可信、可访问性、可维护性几个方面看：注册、查询、购买或提交要形成闭环，用户要能理解入口和结果，弱网、空数据和错误操作要有反馈与恢复路径，账户与业务数据要受到保护，键盘操作和文字对比要被考虑，组件、文档和测试还要能持续维护。需求最好写成可验收的陈述，例如“新用户应能在 3 分钟内完成注册并找到课程报名入口”，而不是只写“网站要方便”。

 徐熠潘

从底层实现看，除了前端、后端和数据库，还需要哪些部分？我们的网站不准备上线，哪些可以先省略？

 GPT

典型访问链路可以理解为“浏览器 → DNS/CDN → 网关或负载均衡 → 前后端服务 → 缓存、数据库、文件存储和消息队列”，周围还可能有认证授权、搜索、日志、监控、部署和安全设施。只做本地演示时，可以省略域名与 DNS、云服务器、HTTPS 证书、CDN、负载均衡、容器编排、API 网关、Redis、消息队列、完整监控告警、CI/CD、WAF/DDoS 防护、高可用灾备和 SEO；但前端交互、必要的本地后端与数据库、权限、输入校验、异常提示、启动说明、Git 协作记录和核心测试仍应保留。

 徐熠潘

也就是说，先按功能复杂度选择最小架构，而不是先把线上基础设施配齐，对吗？

 GPT

对。纯展示或页面复刻可以用前端加静态资源或 JSON；筛选、收藏、主题等单机交互可以使用前端加 `localStorage`；模拟登录、商品详情和购物车可以使用 Mock 数据或本地 JSON；如果有真实注册、角色、后台和持久化 CRUD，再使用本地后端加 MySQL 或 SQLite。`localStorage` 适合演示单机状态，但不能模拟多用户共享数据，清理浏览器数据后也可能丢失内容。

形成的结论

网站质量先按目标任务的完整闭环和可维护性判断；本地演示可以去掉生产承载与运维设施，但不能去掉与功能闭环直接相关的校验、权限、错误处理和测试。后续根据业务复杂度在静态资源、localStorage、Mock 数据和本地后端数据库之间选择最小架构。

1.2 网站设计流程、信息架构与绘图

徐熠潘

网站设计通常应该按照什么流程推进？信息架构和复杂业务逻辑又应该怎样表达？

GPT

可以按“调研与定义 → 信息架构 → 交互设计 → 视觉设计 → 原型验证 → 开发交付 → 测试迭代”推进：先明确目标、用户、角色、范围和约束，再组织栏目、页面、导航、任务流程以及默认、加载、空白、异常和成功状态，之后从低保真线框图到视觉规范和高保真稿，最后交付并从视觉、功能、设备适配和异常处理四方面验收。用户路径回答“用户为什么这样做”，页面路径回答“页面怎样跳转”，两者不能只留下后者。

徐熠潘

如果业务逻辑比较复杂，是否有成熟的绘图规则？一张流程图能不能把所有内容都表达清楚？

GPT

不同问题应使用不同产物：网站地图说明栏目和导航，用户故事或用例说明价值，用户流程图说明操作、决策和页面跳转，BPMN、活动图或泳道图适合多角色交接，UML 时序图说明模块按时间怎样协作，状态图说明对象生命周期，决策表或决策树说明多条件如何得出结果，ER 图说明数据关系。流程回答步骤顺序，规则回答每一步允许或禁止的条件，不能让一张流程图承担两种含义。

徐熠潘

这些图有没有方便的工具？对于课程项目，最少应该先做哪些图？

GPT

网站地图、用户流程、ER 图和 UML 图可以优先使用 diagrams.net (draw.io)；需求讨论和用户旅程适合 FigJam，页面原型和高保真稿适合 Figma，随 Markdown 或代码版本化的图可以用 Mermaid，严格的跨角色审批流可以用 bpmn.io。课程项目的最小设计集是网站地图、核心用户流程、ER 图和角色权限表；遇到审核或订单逻辑，再补 BPMN、状态图、决策表或时序图。

形成的结论

先以用户、目标和任务闭环建立信息架构，再选择与问题匹配的图。最小设计集固定为网站地图、核心用户流程、ER 图和角色权限表，复杂度增加时按需补充 BPMN、状态图、决策表和时序图。

1.3 接口契约与组件协作

徐熠潘

怎样确定接口规范，才能让各组件保留设计自由度，又能稳定组合？

GPT

关键不是把接口写得越细越好，而是建立稳定、最小、可验证、可演进的接口契约，让组件依赖契约而不是彼此的内部实现。一个完整契约至少约定五层：语法（地址、方法、字段和格式）、数据（类型、必填性、范围和默认值）、语义（操作在业务上代表什么）、行为（成功、失败、重复请求和冲突的结果），以及权限、分页、幂等、性能和限流等非功能约束。

徐熠潘

这是否意味着应该先做 API First？接口变更时，怎样判断还兼容？

GPT

可以采用 API First：先明确业务规则并设计 OpenAPI 规范，前端基于 Mock 数据并行开发，后端按规范实现，再用自动化测试验证契约。理想上旧客户端可发送的请求集合满足 $R_{old} \subseteq R_{new}$ ，但还要检查响应语义、权限、状态码和副作用。新增接口或响应中的可选字段通常是兼容变化；删除字段、改变字段类型或语义、把可选改为必填、改变错误码含义，则是不兼容变化，重大变化使用例如 `/api/v1/` 和 `/api/v2/` 的版本化接口。

徐熠潘

除了接口文档，还应该怎样验证契约没有漂移？

GPT

至少覆盖接口单元测试、集成测试和契约测试。对于审核、订单等复杂对象，接口文档还应与状态图、决策表和时序图配套，分别验证状态能否转换、条件如何判断、模块怎样协作；这样可以同时检查字段格式和真实业务行为。

形成的结论

接口以最小、稳定、可验证的契约作为协作边界，优先采用 OpenAPI First，并把语义、权限、状态码和副作用纳入兼容性判断。接口单元测试、集成测试和契约测试共同防止前后端实现偏离；重大不兼容变化采用版本化接口。

1.4 WEMOVE 面向谁、完成什么、项目目标是什么

 徐熠潘

重构后的 WEMOVE 网站主要面向哪些用户？既然产品服务儿童，是否需要创建儿童账户？

 GPT

目标用户是成年人，主要包括家长与成年消费者、礼品购买者、零售商、批发商、进口商及采购人员、教育或活动机构采购人员，以及已合作或潜在经销商。产品可以服务儿童运动、协调能力和亲子互动，但账户注册、交易和个人信息收集应面向成年人：不创建儿童账户，也不主动收集儿童个人信息。

 徐熠潘

这些用户进入网站后要完成什么？除了浏览和购买，还要支持哪些任务？

 GPT

任务可以归为五类：浏览与查询产品、文章、FAQ、下载资料和经销商；收藏、加购、结算、查订单及申请退换等购买与售后，未开通零售时转为“在哪里购买”或联系咨询；经销商申请、专属目录、价格、库存、交期、询价和批量下单等商务合作；阅读玩法和安全信息、下载说明书或证书、提交联系与售后表单、订阅邮件等内容与支持；以及后台维护商品、价格、订单、经销商、文章、Banner、FAQ、SEO 和报表。前台主链路是“了解 → 比较或咨询 → 购买或合作”，后台主链路是“维护 → 运营 → 分析”。

 徐熠潘

所以项目的最终目标不是把旧页面原样复刻，而是把品牌站变成可以持续运营的业务门户？

 GPT

对。目标是展示品牌、产品价值、玩法和安全信息，同时支持 B2C 消费者转化与 B2B 经销商合作；后台应能在不改代码的情况下维护产品、内容和经销商信息，并按角色隔离价格、订单和资料权限。系统还应考虑搜索优化、行为分析、响应式、多语言、权限审计、安全保护和可扩展的数据结构。

 形成的结论

WEMOVE 的账户、交易和信息收集面向成年人；功能设计围绕“了解、比较或咨询、购买或合作”的前台闭环，以及“维护、运营、分析”的后台闭环。项目目标是可持续运营的业务门户，而不是只完成旧页面的静态复刻。

1.5 需求文档与 SRS 模板

徐熠潘

如果要为网站设计需求文档，有哪些模板可以参考？不同模板分别解决什么问题？

GPT

不宜只套用一种模板。PRD 主要回答为什么做、为谁做、做什么和优先级；网站功能需求说明回答有哪些页面、每页能做什么；BRD 说明业务痛点、流程变化和成功标准，适合经销商、订单和审核等协作；SRS 则把系统必须如何工作、异常与质量约束以及验收方式写清楚，适合开发前定稿和课程交付。

徐熠潘

《requirements-doc-template.pdf》这种 IEEE 风格的需求文档，主体通常包括哪些内容？

GPT

它是基于 IEEE 830-1998 改编的 SRS 模板，主线是把“为什么做、做什么、约束什么、如何验证”串起来。主体包括引言、总体描述、按编号编写的具体功能和质量需求、用例图、领域类图、数据字典、典型场景与时序图、状态图、V1/V2 原型，以及参考资料和负责人联系方式。对当前项目，最值得复用的是“具体需求 + 用例或场景 + 状态图 + 原型”，让自然语言、模型、操作验证和测试彼此衔接。

徐熠潘

那份现代 Markdown 的 SRS-Template 应该怎样使用？是否需要把实现方式也写进需求？

GPT

该目录按 IEEE 830 与 ISO/IEC/IEEE 29148 的结构提供了完整模板、简洁骨架和单条需求模板，例如 `srs-template.md`、`srs-template-bare.md`、`req-template.md` 和 `req-template-bare.md`。它的边界是写清系统必须做什么，实现方式交给架构设计和代码文档；最终 SRS 应覆盖接口、功能、质量和合规需求、验证方式，并建立需求 ID 到测试证据的追踪关系。

形成的结论

需求文档按阶段组合使用：PRD 或 BRD 对齐目标和业务，网站功能说明组织页面，SRS 固化可实现、可测试、可验收的范围与约束。实现方案不提前塞进需求模板，但需求应能追踪到模型、原型和测试证据。

1.6 可维护性、模块边界与技术栈

徐熠潘

从技术栈角度，怎样保证“品牌官网 + B2C + 经销商门户 + 管理后台”后续还能维护？

GPT

可维护性的核心不是追逐热门框架，而是控制变化传播范围，可以粗略理解为 $\text{维护成本} \propto \text{模块耦合度} \times \text{需求变更频率}$ 。因此要降低耦合，建立唯一数据来源，并让需求、接口、代码和测试能够追踪。针对这类业务复杂度，讨论中更适合先采用模块化单体，而不是一开始拆成微服务：应用保持一个可部署、可调试的整体，内部按产品、订单、经销商、内容、身份等业务域划分模块，每个模块只公开必要能力。

徐熠潘

讨论中给出的参考技术选型是什么？它分别解决哪些维护问题？

GPT

参考方案是 React 加 TypeScript 负责前台和后台，NestJS 加 TypeScript 按业务域组织后端，PostgreSQL 配合迁移脚本管理关系数据，图片和说明书等非结构化文件进入对象存储，耗时工作进入 Redis 队列。这个选型强调共享组件与类型、隔离发布和权限边界、让数据库结构可复现，并避免慢任务阻塞交互；它是针对目标网站规模的参考方案，不是必须替换当前课程仓库的命令。

徐熠潘

那当前共享仓库已经约定的 Vue 3、Java 21、Spring Boot 和 MySQL 是否需要被替换？

GPT

不需要。当前共享实现已经约定 Vue 3 加 TypeScript、Java 21 加 Spring Boot 和 MySQL，继续开发时应优先遵循这些技术约束。可以把讨论中的原则迁移到既定技术栈：按业务域组织模块，限制跨模块依赖，使用共享类型或组件，保留数据库迁移、唯一数据源、接口契约和自动化测试。

形成的结论

当前方向是“既定技术栈内的模块化单体”：不因参考选型而整体换栈，把可维护性落实为业务域边界、必要能力公开、数据库迁移、唯一数据源、接口契约和测试追踪。React/NestJS/PostgreSQL/Redis 只作为规模化设计的参考，不改变仓库既有 Vue 3、Java 21、Spring Boot 和 MySQL 约束。